

Python Notes

Special Methods (Dunder Methods) and Inheritance

1 Revision: Everything is an Object

In the previous classes, we learned:

```
name = "Ram"  
age = 20  
marks = [45, 67, 89]
```

We treated them simply as variables.
But behind the scenes:

- String is an object
- Integer is an object
- List is an object
- Dictionary is an object

Everything in Python is an object.
Objects are created from classes.

Class $\rightarrow$ Object

Example:

```
name = "Ram"
```

Actually:

```
name = str("Ram")
```

Python creates a string object from the `str` class.

2 Special Methods (Dunder Methods)

Some methods in Python have names like:

```
__init__  
__str__  
__add__  
__sub__  
__mul__
```

Notice the double underscore before and after the name.
These are called:

Special Methods

or

Dunder Methods

(Dunder = Double Under)
or

Magic Methods

Python automatically calls these methods in special situations.

3 The Matrix Class

Suppose we create our own Matrix class.

```
class Matrix:
    def __init__(self, a, b):
        self.a = a
        self.b = b
```

Creating objects:

```
a1 = Matrix(5, 20)
a2 = Matrix(3, 4)
```

4 __init__ Method

`__init__()` is called automatically when an object is created.

```
a1 = Matrix(5, 20)
```

Python internally does:

```
a1.__init__(5, 20)
```

Its job is to initialize object data.

5 The Problem with print()

Suppose we write:

```
print(a1)
```

Output:

```
<__main__.Matrix object at 0x7f....>
```

Not very useful.

We want something better.

6 `__str__` Method

`__str__()` tells Python how an object should look when printed.

```
def __str__(self):
    matrix_str_format = f"""
    | {self.a} |
    | {self.b} |
    """
    return matrix_str_format
```

Now:

```
print(a1)
```

Output:

```
| 5 |
| 20 |
```

Much nicer.

7 Operator Overloading

Normally:

```
5 + 3
```

works because integers know how to add themselves.

But what about:

```
a1 + a2
```

Python doesn't know how to add two Matrix objects.

We must teach Python.

8 `__add__` Method

```
def __add__(self, other):
    new_a = self.a + other.a
    new_b = self.b + other.b

    return Matrix(new_a, new_b)
```

Now:

```
a1 = Matrix(5, 20)
a2 = Matrix(3, 4)
```

```
print(a1 + a2)
```

Output:

```
| 8 |
| 24 |
```

Python converts:

```
a1 + a2
into:
a1.__add__(a2)
```

9 Validating Input

What if someone writes:

```
a1 + 2
```

This should not be allowed.

```
def __add__(self, other):
    if not isinstance(other, Matrix):
        raise Exception(
            "Second operand must be a valid matrix"
        )

    return Matrix(
        self.a + other.a,
        self.b + other.b
    )
```

Now Python raises an error with a meaningful message.

10 __sub__ Method

Used when subtraction occurs.

```
def __sub__(self, other):
    return Matrix(
        self.a - other.a,
        self.b - other.b
    )
```

Example:

```
print(a1 - a2)
```

Python converts it to:

```
a1.__sub__(a2)
```

11 __mul__ Method

Used when multiplication occurs.

```
def __mul__(self, other):
    return Matrix(
        self.a * other,
        self.b * other
    )
```

Example:

```
print(a1 * 6)
```

Python internally does:

```
a1.__mul__(6)
```

Output:

```
| 30 |  
| 120 |
```

12 What is Operator Overloading?

Giving special meaning to operators for our custom classes.

Examples:

```
+  
-  
*  
/  
==  
<  
>
```

can all be customized.

Examples:

```
__add__  
__sub__  
__mul__  
__eq__  
__lt__  
__gt__
```

13 Inheritance

Suppose we create a User class.

```
class User:  
    def __init__(self,  
                  username,  
                  password,  
                  email="example@ex.com"):  
  
        self.username = username  
  
        if len(password) < 6:  
            raise Exception(  
                "Password must be at least 6 chars long"  
            )
```

```

        if username in password:
            raise Exception(
                "Password cannot contain username"
            )

        self.password = password

```

14 Creating User Objects

```
u1 = User("ram", "ram@123")
```

This user now has:

- username
- password validation

15 __str__ in User

```

def __str__(self):
    return f"{self.username}'s account"

```

Now:

```
print(u1)
```

Output:

```
ram's account
```

16 The Repetition Problem

Suppose we need:

- Student
- Teacher
- Admin

All need:

- username
- password
- email

Should we copy the same code again and again?

No.

That would create duplication.

17 Inheritance to the Rescue

```
class Student(User):  
    pass
```

Student automatically receives:

- username
- password
- validations
- methods

from User.

18 Parent and Child Class

User

is called:

- Parent Class
- Base Class
- Super Class

Student

is called:

- Child Class
- Derived Class
- Sub Class

19 Adding Extra Data

```
class Student(User):  
  
    def __init__(  
        self,  
        username,  
        password,  
        father_name,  
        email="opt@opt.com"  
    ):
```

```

        super().__init__(
            username,
            password,
            email
        )

        self.father_name = father_name

```

20 What is super()?

```

super().__init__(
    username,
    password,
    email
)

```

calls the parent class constructor.

Meaning:

```

User.__init__(
    self,
    username,
    password,
    email
)

```

This allows us to reuse parent logic.

21 Creating Student Objects

```

s1 = Student(
    "ram",
    "ram@123",
    "ram's father"
)

```

Student now contains:

- username
- password
- father_name

22 Teacher Class


```
class Teacher(User):

    def __init__(self,
        username,
        password,
        salary,
        email="opt@opt.com"
    ):

        super().__init__(
            username,
            password,
            email

        )

        self.salary = salary
```

Usage:

```
t1 = Teacher(
    "hari",
    "ASDF0123",
    45000
)
```

23 Method Overriding

When a child class writes its own version of a method already present in parent class, it is called:

Method Overriding

Example:

```
class Student(User):

    def __str__(self):
        return "Student Account"
```

Now Student uses its own version.

24 Real Django Examples

Inheritance is everywhere in Django.

Models

```
class Student(models.Model):
    name = models.CharField(...)
    address = models.CharField(...)
```

Student inherits from:

```
models.Model
```

Because of this, Student automatically gets:

```
save()
delete()
objects.create()
filter()
all()
```

and many other features.

Forms

```
class StudentForm(forms.ModelForm):
    pass
```

StudentForm inherits from:

```
forms.ModelForm
```

and automatically receives:

```
is_valid()
save()
errors
```

and many more features.

25 ORM

ORM stands for:

Object Relational Mapping

It converts Python objects into database records.

Example:

```
Student.objects.create(
    name="Ram",
    address="KTM"
)
```

Instead of writing SQL manually.

26 Virtual Environment

As projects grow, different projects may need different package versions.

Example:

Project A:

Django 4.2

Project B:

Django 5.2

Virtual Environment allows each project to have its own isolated packages.

Common commands:

```
python -m venv venv
```

Activate:

Linux/Mac:

```
source venv/bin/activate
```

Windows:

```
venv\Scripts\activate
```

Install packages:

```
pip install django
```

27 Key Takeaways

1. Methods like `__init__`, `__str__`, `__add__` are called Dunder (Magic) Methods.
2. Python automatically calls these methods in special situations.
3. Operator Overloading allows operators like `+`, `-`, `*` to work with our own classes.
4. Inheritance helps reuse existing code.
5. Parent class provides features to child classes.
6. `super()` calls parent class methods.
7. Django heavily relies on inheritance.
8. ORM converts Python objects into database operations.
9. Virtual Environments keep project dependencies isolated.

28 Practice Questions

Basic

1. What are Dunder Methods?
2. Why is `__init__()` called automatically?
3. What is the purpose of `__str__()`?
4. What is operator overloading?
5. What is inheritance?
6. What is a Parent Class?

7. What is a Child Class?
8. What does `super()` do?
9. What does ORM stand for?
10. Why do we use virtual environments?

Coding Practice

1. Create a `Point` class with x and y coordinates and implement `__str__()`.
2. Add `__add__()` to add two `Point` objects.
3. Create a `Rectangle` class and overload multiplication to scale its size.
4. Create a `Person` class and derive `Student` from it.
5. Create an `Employee` class inheriting from `Person`.
6. Override `__str__()` inside `Student`.
7. Create a `Vehicle` parent class and derive `Car` and `Bike`.
8. Create a `Shape` parent class and derive `Circle` and `Rectangle`.